Robotics and its Application (AI352) | Assignment 4

Forward and Inverse Kinematics of a Manipulator

Student Name: *Naishadh Rana*

Roll No: U23CS014

Part A: Forward and Inverse Kinematics for 2R Planar Robot Arm

A **2R planar robot arm** consists of two revolute joints and two rigid links. Understanding the relationship between joint angles and end-effector position is fundamental to robot control.

Forward Kinematics (FK)

Forward kinematics computes the end-effector position (x, y) given joint angles (θ_1, θ_2) and link lengths (L_1, L_2).

Mathematical Formulation:

$$x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2)$$

$$y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2)$$

Inverse Kinematics (IK)

Inverse kinematics finds joint angles (θ_1, θ_2) given a desired end-effector position (x, y).

Derivation using the Law of Cosines:

From the geometry, the distance from origin to end-effector is:

$$r^2 = x^2 + y^2 = L_1^2 + L_2^2 + 2L_1L_2 \cos(\theta_2)$$

Solving for θ_2:

$$\cos(\theta_2) = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}$$

$$\theta_2 = \pm \arccos\left(\frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

The two solutions correspond to **elbow-up** (+) and **elbow-down** (-) configurations.

For θ_1:

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin(\theta_2), L_1 + L_2 \cos(\theta_2))$$

Reachability Condition: A point (x, y) is reachable if:

$$|L_1 - L_2| \leq \sqrt{x^2 + y^2} \leq L_1 + L_2$$

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation
from IPython.display import HTML
```

In [ ]:
```
def forward_kinematics_2r(theta1, theta2, L1, L2):
    x = L1 * np.cos(theta1) + L2 * np.cos(theta1 + theta2)
    y = L1 * np.sin(theta1) + L2 * np.sin(theta1 + theta2)
    return x, y


def inverse_kinematics_2r(x, y, L1, L2):
    d_sq = x**2 + y**2
    d = np.sqrt(d_sq)

    if d > L1 + L2 or d < abs(L1 - L2):
        print(f"Point ({x:.2f}, {y:.2f}) is outside the reachable workspace!")
        return []

    cos_theta2 = (d_sq - L1**2 - L2**2) / (2 * L1 * L2)
    cos_theta2 = np.clip(cos_theta2, -1.0, 1.0)

    solutions = []

    theta2_down = np.arccos(cos_theta2)
    theta1_down = np.arctan2(y, x) - np.arctan2(L2 * np.sin(theta2_down), L1 + 
    solutions.append((theta1_down, theta2_down))

    theta2_up = -np.arccos(cos_theta2)
    theta1_up = np.arctan2(y, x) - np.arctan2(L2 * np.sin(theta2_up), L1 + L2 *
    solutions.append((theta1_up, theta2_up))

    return solutions


def plot_robot_2r(theta1, theta2, L1, L2, ax=None, title="2R Robot Configuratio
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 8))

    x0, y0 = 0, 0
    x1 = L1 * np.cos(theta1)
    y1 = L1 * np.sin(theta1)
    x2 = x1 + L2 * np.cos(theta1 + theta2)
    y2 = y1 + L2 * np.sin(theta1 + theta2)

    ax.plot([x0, x1], [y0, y1], 'o-', lw=4, markersize=10, color=color, label=f
    ax.plot([x1, x2], [y1, y2], 's-', lw=4, markersize=10, color=color, alpha=0
    ax.plot(x2, y2, 'r*', markersize=15, label=f'End-effector ({x2:.2f}, {y2:.2

    ax.add_patch(plt.Circle((0, 0), L1 + L2, fill=False, linestyle='--', color=
```

```
    ax.add_patch(plt.Circle((0, 0), abs(L1 - L2), fill=False, linestyle='--', c

    ax.set_xlim(-(L1 + L2) * 1.3, (L1 + L2) * 1.3)
    ax.set_ylim(-(L1 + L2) * 1.3, (L1 + L2) * 1.3)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.3)
    ax.set_xlabel('X (m)')
    ax.set_ylabel('Y (m)')
    ax.set_title(title)
    ax.legend(loc='upper right')

    return ax
```

Validation and Visualization

We demonstrate both functions with:

- Link lengths: $L_1 = 3$ m, $L_2 = 2$ m
- Workspace: Inner radius = $|3 - 2| = 1$ m, Outer radius = $3 + 2 = 5$ m

Test Cases:

1. Point $(4, 2)$ - Inside workspace
2. Point $(3, 3)$ - Inside workspace
3. Point $(1.5, 0.5)$ - Near inner boundary

In [ ]:
```
# Define link lengths
L1, L2 = 3.0, 2.0

# Test points within workspace
test_points = [(4, 2), (3, 3), (1.5, 0.5)]

fig, axes = plt.subplots(2, 3, figsize=(15, 10))

for idx, (x_target, y_target) in enumerate(test_points):
    print(f"\n{'='*60}")
    print(f"Test Case {idx + 1}: Target point ({x_target}, {y_target})")
    print(f"{'='*60}")

    # Inverse Kinematics
    solutions = inverse_kinematics_2r(x_target, y_target, L1, L2)

    for sol_idx, (theta1, theta2) in enumerate(solutions):
        config_name = "Elbow-Down" if sol_idx == 0 else "Elbow-Up"

        # Forward Kinematics validation
        x_computed, y_computed = forward_kinematics_2r(theta1, theta2, L1, L2)

        print(f"\n{config_name} Configuration:")
        print(f"  θ₁ = {np.degrees(theta1):.2f}° ({theta1:.4f} rad)")
        print(f"  θ₂ = {np.degrees(theta2):.2f}° ({theta2:.4f} rad)")
        print(f"  FK Validation: ({x_computed:.4f}, {y_computed:.4f})")
        print(f"  Error: ({abs(x_target - x_computed):.6f}, {abs(y_target - y_c

        # Plot configuration
        ax = axes[sol_idx, idx]
        plot_robot_2r(theta1, theta2, L1, L2, ax=ax,
```

```
                        title=f"Point ({x_target}, {y_target}) - {config_name}\nθ
                        color='blue' if sol_idx == 0 else 'green')
        ax.plot(x_target, y_target, 'ko', markersize=8)

plt.tight_layout()
plt.show()
```

Extension to n-R Planar Robot Arm

For an **n-R planar robot** with n revolute joints and link lengths $L_1, L_2, \ldots, L_n$:

Forward Kinematics:

$$x = \sum_{i=1}^{n} L_i \cos\left(\sum_{j=1}^{i} \theta_j\right)$$

$$y = \sum_{i=1}^{n} L_i \sin\left(\sum_{j=1}^{i} \theta_j\right)$$

Inverse Kinematics Challenge:

- For $n > 2$, the robot is **kinematically redundant** (infinite solutions for a given point)
- Common approaches:
  1. **Jacobian Pseudo-inverse** method
  2. **Cyclic Coordinate Descent (CCD)**
  3. **FABRIK** (Forward And Backward Reaching Inverse Kinematics)
  4. **Gradient Descent** optimization

I have tried implementing the **Jacobian Pseudo-inverse** method for numerical IK.

In [ ]:
```
def forward_kinematics_nr(thetas, link_lengths):
    thetas = np.array(thetas)
    link_lengths = np.array(link_lengths)

    cumulative_angles = np.cumsum(thetas)
    x_positions = np.cumsum(link_lengths * np.cos(cumulative_angles))
    y_positions = np.cumsum(link_lengths * np.sin(cumulative_angles))

    X = np.concatenate([[0], x_positions])
    Y = np.concatenate([[0], y_positions])

    return X[-1], Y[-1], (X, Y)


def compute_jacobian_nr(thetas, link_lengths):
    n = len(thetas)
    J = np.zeros((2, n))
    cumulative_angles = np.cumsum(thetas)

    for i in range(n):
        for j in range(i, n):
            J[0, i] -= link_lengths[j] * np.sin(cumulative_angles[j])
```

```
            J[1, i] += link_lengths[j] * np.cos(cumulative_angles[j])

    return J


def inverse_kinematics_nr(x_target, y_target, link_lengths, initial_thetas=None
                          max_iterations=1000, tolerance=1e-6, damping=0.1):
    n = len(link_lengths)
    link_lengths = np.array(link_lengths)

    max_reach = np.sum(link_lengths)
    target_dist = np.sqrt(x_target**2 + y_target**2)

    if target_dist > max_reach:
        print(f"Target ({x_target:.2f}, {y_target:.2f}) is outside maximum reac
        return None, False

    if initial_thetas is None:
        thetas = np.zeros(n)
    else:
        thetas = np.array(initial_thetas).copy()

    for iteration in range(max_iterations):
        x_curr, y_curr, _ = forward_kinematics_nr(thetas, link_lengths)
        error = np.array([x_target - x_curr, y_target - y_curr])
        error_magnitude = np.linalg.norm(error)

        if error_magnitude < tolerance:
            return thetas, True

        J = compute_jacobian_nr(thetas, link_lengths)
        JJT = J @ J.T
        damped_inverse = J.T @ np.linalg.inv(JJT + damping**2 * np.eye(2))
        delta_theta = damped_inverse @ error

        thetas += delta_theta
        thetas = np.arctan2(np.sin(thetas), np.cos(thetas))

    return thetas, False


def plot_robot_nr(thetas, link_lengths, ax=None, title="n-R Robot Configuration
    if ax is None:
        fig, ax = plt.subplots(figsize=(8, 8))

    _, _, (X, Y) = forward_kinematics_nr(thetas, link_lengths)

    ax.plot(X, Y, 'o-', lw=3, markersize=8, color='blue')
    ax.plot(X[-1], Y[-1], 'r*', markersize=15, label=f'End-effector ({X[-1]:.2f
    ax.plot(0, 0, 'ko', markersize=10, label='Base')

    max_reach = np.sum(link_lengths)
    ax.add_patch(plt.Circle((0, 0), max_reach, fill=False, linestyle='--', colo

    ax.set_xlim(-max_reach * 1.2, max_reach * 1.2)
    ax.set_ylim(-max_reach * 1.2, max_reach * 1.2)
    ax.set_aspect('equal')
    ax.grid(True, alpha=0.3)
    ax.set_xlabel('X (m)')
    ax.set_ylabel('Y (m)')
```

```
    ax.set_title(title)
    ax.legend()

    return ax
```

Interactive n-R Robot Demo

The user can input the number of joints n and link lengths. The program computes IK for sample target points and validates using FK.

In [ ]:
```
def demo_nr_robot(n, link_lengths, target_points):
    print(f"\n{'='*50}")
    print(f"n-R PLANAR ROBOT (n = {n})")
    print(f"Link lengths: {link_lengths}")
    print(f"Maximum reach: {np.sum(link_lengths):.2f} m")
    print(f"{'='*50}")

    fig, axes = plt.subplots(1, len(target_points), figsize=(5*len(target_point
    if len(target_points) == 1:
        axes = [axes]

    for idx, (x_t, y_t) in enumerate(target_points):
        print(f"\n--- Target {idx + 1}: ({x_t}, {y_t}) ---")
        thetas, success = inverse_kinematics_nr(x_t, y_t, link_lengths)

        if success:
            x_computed, y_computed, _ = forward_kinematics_nr(thetas, link_leng

            print(f"IK Solution (radians): {np.round(thetas, 4)}")
            print(f"IK Solution (degrees): {np.round(np.degrees(thetas), 2)}")
            print(f"FK Validation: ({x_computed:.4f}, {y_computed:.4f})")
            print(f"Position Error: {np.sqrt((x_t-x_computed)**2 + (y_t-y_compu

            plot_robot_nr(thetas, link_lengths, ax=axes[idx], title=f"Target ({
            axes[idx].plot(x_t, y_t, 'go', markersize=12, label='Target', zorde
            axes[idx].legend()
        else:
            print("IK failed to converge!")
            axes[idx].set_title(f"Target ({x_t}, {y_t}) - Failed")

    plt.tight_layout()
    plt.show()


n = 4
link_lengths = [2.0, 1.5, 1.0, 0.5]
target_points = [(4.0, 1.0), (2.0, 3.0), (0.5, 0.5)]

demo_nr_robot(n, link_lengths, target_points)
```

In [ ]:
```
n_user = int(input("Enter number of joints (n): "))
link_lengths_user = []
for i in range(n_user):
    length = float(input(f"Enter length of link {i+1} (m): "))
    link_lengths_user.append(length)

x_target = float(input("Enter target X coordinate (m): "))
y_target = float(input("Enter target Y coordinate (m): "))
```

```
target_user = (x_target, y_target)

print(f"\nNumber of joints: {n_user}")
print(f"Link lengths: {link_lengths_user}")
print(f"Target position: {target_user}")

demo_nr_robot(n_user, link_lengths_user, [target_user])
```

Part B: Rotation Matrices and Coordinate Transformations

Theory: Rotation Matrices

Basic Rotation Matrices about Fixed Axes:

Roll (Rotation about X-axis):

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix}$$

Pitch (Rotation about Y-axis):

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

Yaw (Rotation about Z-axis):

$$R_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Composite Rotation (Fixed Axes - Pre-multiply):

For sequential rotations about fixed axes: Yaw → Pitch → Roll

$$R_{composite} = R_x(\text{roll}) \cdot R_y(\text{pitch}) \cdot R_z(\text{yaw})$$

> **Note:** For fixed-axis rotations, we pre-multiply (right-to-left order corresponds to first-to-last rotation)

In [ ]:
```
def Rx(phi):
    c, s = np.cos(phi), np.sin(phi)
    return np.array([
        [1, 0, 0],
        [0, c, -s],
        [0, s, c]
    ])

def Ry(theta):
    c, s = np.cos(theta), np.sin(theta)
```

```python
    return np.array([
        [c, 0, s],
        [0, 1, 0],
        [-s, 0, c]
    ])

def Rz(psi):
    c, s = np.cos(psi), np.sin(psi)
    return np.array([
        [c, -s, 0],
        [s, c, 0],
        [0, 0, 1]
    ])

def print_matrix(M, name="Matrix", precision=4):
    print(f"\n{name}:")
    for row in M:
        print("  [", end="")
        print("  ".join(f"{val:8.{precision}f}" for val in row), end="")
        print("]")
```

Problem B.1: Composite Rotation Matrix (UAV Drone)

Problem: A drone performs sequential rotations about fixed axes:

1. Yaw of $\pi/2$ about Z
2. Pitch of $-\pi/2$ about Y
3. Roll of $\pi/2$ about X

Find the composite rotation matrix.

Step-by-Step Solution:

Step 1: Compute individual rotation matrices.

$$R_z(\pi/2) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_y(-\pi/2) = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

$$R_x(\pi/2) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

Step 2: For fixed-axis rotations, pre-multiply in reverse order:

$$R_{composite} = R_x(\pi/2) \cdot R_y(-\pi/2) \cdot R_z(\pi/2)$$

Step 3: Compute intermediate product $R_y(-\pi/2) \cdot R_z(\pi/2)$:

$$R_y \cdot R_z = \begin{bmatrix} 0 & 0 & -1 \\ 1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

Step 4: Compute final product $R_x(\pi/2) \cdot (R_y \cdot R_z)$:

$$R_{composite} = \begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Verification: $\det(R) = 1$ and $R^T R = I$ ✓

In [ ]:
```
# Define angles
yaw, pitch, roll = np.pi/2, -np.pi/2, np.pi/2

# Compute composite rotation (fixed axes: pre-multiply in reverse order)
R_composite = Rx(roll) @ Ry(pitch) @ Rz(yaw)

print_matrix(R_composite, "R_composite")
print(f"\nDeterminant: {np.linalg.det(R_composite):.4f}")
```

Problem B.2: Coordinate Transformation

Problem: A point P at the tool tip has mobile coordinates $[p]^M = [0, 0, 0.6]^T$. After applying:

1. Yaw of 45° about Z
2. Pitch of 60° about Y
3. Roll of 90° about X

Find the fixed-frame coordinates $[p]^F$.

Step-by-Step Solution:

Step 1: Convert angles to radians.

- Yaw: $45^\circ = \pi/4$ rad
- Pitch: $60^\circ = \pi/3$ rad
- Roll: $90^\circ = \pi/2$ rad

Step 2: Compute composite rotation matrix (fixed axes → pre-multiply in reverse):

$$R_{composite} = R_x(90^\circ) \cdot R_y(60^\circ) \cdot R_z(45^\circ)$$

Step 3: Compute $R_z(45^\circ)$:

$$R_z = \begin{bmatrix} 0.7071 & -0.7071 & 0 \\ 0.7071 & 0.7071 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 4: Compute $R_y(60^\circ)$:

$$R_y = \begin{bmatrix} 0.5 & 0 & 0.866 \\ 0 & 1 & 0 \\ -0.866 & 0 & 0.5 \end{bmatrix}$$

Step 5: Compute $R_x(90^\circ)$:

$$R_x = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

Step 6: Multiply to get $R_{composite}$ and transform point:

$$[p]^F = R_{composite} \cdot [p]^M = R_{composite} \cdot \begin{bmatrix} 0 \\ 0 \\ 0.6 \end{bmatrix}$$

Step 7: The third column of $R_{composite}$ scaled by 0.6 gives us $[p]^F$:

$$[p]^F = \begin{bmatrix} 0.5196 \\ -0.3 \\ 0 \end{bmatrix}$$

In [ ]:
```
# Mobile coordinates
p_mobile = np.array([0, 0, 0.6])

# Angles in radians
yaw, pitch, roll = np.radians(45), np.radians(60), np.radians(90)

# Composite rotation (fixed axes: pre-multiply in reverse order)
R_composite = Rx(roll) @ Ry(pitch) @ Rz(yaw)

# Transform point
p_fixed = R_composite @ p_mobile

print(f"Mobile coordinates [p]^M = {p_mobile}")
print(f"\nFixed-frame coordinates [p]^F:")
print(f"  x = {p_fixed[0]:.4f}")
print(f"  y = {p_fixed[1]:.4f}")
print(f"  z = {p_fixed[2]:.4f}")
```

Problem B.3: Composite Transformation

Problem: Find the composite transformation matrix for:

1. Translation of M along f_2 (Y-axis) by 3 units
2. Rotation of M about f_3 (Z-axis) by 180°

Step-by-Step Solution:

Step 1: Write the translation matrix $T_{trans}(0, 3, 0)$:

$$T_{trans} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2: Write the rotation matrix $T_{rot,z}(180^\circ)$:

Since $\cos(180^\circ) = -1$ and $\sin(180^\circ) = 0$:

$$T_{rot,z} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 3: Compute composite (translation first, then rotation):

$$T_{composite} = T_{rot,z}(180^\circ) \cdot T_{trans}(0, 3, 0)$$

Step 4: Multiply matrices:

$$T_{composite} = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & -3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Interpretation: The composite transformation rotates 180° about Z and the translation becomes $(0, -3, 0)$ due to the rotation.

In [ ]:
```
def Tx(d):
    return np.array([
        [1, 0, 0, d],
        [0, 1, 0, 0],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ], dtype=float)

def Ty(d):
    return np.array([
        [1, 0, 0, 0],
        [0, 1, 0, d],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ], dtype=float)

def Tz(d):
    return np.array([
        [1, 0, 0, 0],
        [0, 1, 0, 0],
        [0, 0, 1, d],
        [0, 0, 0, 1]
    ], dtype=float)

def Rx_h(phi):
    c, s = np.cos(phi), np.sin(phi)
    return np.array([
```

```
        [1, 0, 0, 0],
        [0, c, -s, 0],
        [0, s, c, 0],
        [0, 0, 0, 1]
    ])

def Ry_h(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([
        [c, 0, s, 0],
        [0, 1, 0, 0],
        [-s, 0, c, 0],
        [0, 0, 0, 1]
    ])

def Rz_h(psi):
    c, s = np.cos(psi), np.sin(psi)
    return np.array([
        [c, -s, 0, 0],
        [s, c, 0, 0],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

def print_matrix_4x4(M, name="Matrix"):
    print(f"\n{name}:")
    for row in M:
        print("  [", end="")
        print("  ".join(f"{val:8.4f}" for val in row), end="")
        print("]")
```

In [ ]:

```
# Translation along Y by 3 units
T_trans = Ty(3)

# Rotation about Z by 180°
T_rot = Rz_h(np.pi)

# Composite: translation first, then rotation
T_composite = T_rot @ T_trans

print_matrix_4x4(T_composite, "T_composite")
```

Problem B.4: Complex Transformation

Problem: Find the composite homogeneous transformation matrix for the sequence:

1. Yaw of $-\pi/2$ about Z
2. Translation of 10 cm along X
3. Pitch of $\pi/2$ about Y
4. Translation of 20 cm along Z
5. Roll of $\pi/2$ about X
6. Translation of 30 cm along Y

Step-by-Step Solution:

Step 1: Define each transformation matrix.

$T_1 = R_z(-\pi/2)$: Yaw -90°

$$T_1 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$T_2 = T_x(0.1)$: Translation 10 cm along X

$$T_2 = \begin{bmatrix} 1 & 0 & 0 & 0.1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$T_3 = R_y(\pi/2)$: Pitch 90°

$$T_3 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$T_4 = T_z(0.2)$: Translation 20 cm along Z

$$T_4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$T_5 = R_x(\pi/2)$: Roll 90°

$$T_5 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$T_6 = T_y(0.3)$: Translation 30 cm along Y

$$T_6 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0.3 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Step 2: Compute composite transformation:

$$T_{composite} = T_1 \cdot T_2 \cdot T_3 \cdot T_4 \cdot T_5 \cdot T_6$$

Step 3: The final matrix encodes:

- Rotation part: Combined orientation from all rotations
- Translation part: Accumulated displacement (in meters)

In [ ]:

```
# Define each transformation
T1 = Rz_h(-np.pi/2)   # Yaw -π/2
T2 = Tx(10)         # Translation 10 cm along X
T3 = Ry_h(np.pi/2)    # Pitch π/2
T4 = Tz(20)         # Translation 20 cm along Z
T5 = Rx_h(np.pi/2)    # Roll π/2
T6 = Ty(30)         # Translation 30 cm along Y

# Composite transformation
T_composite = T1 @ T2 @ T3 @ T4 @ T5 @ T6

print_matrix_4x4(T_composite, "T_composite")

# Extract components
R_final = T_composite[:3, :3]
t_final = T_composite[:3, 3]

print(f"\nTranslation: [{t_final[0]:.1f}, {t_final[1]:.1f}, {t_final[2]:.1f}] cm
```